

Fall Detection: Progress Presentation

Abbos Aliboev

Table of contents

01

Motivation

02

System Overview

03

Dataset

04

Pose Completion + CNN + TE

05

Model Architecture

06

Comparative Table

07

Develop. Progress

08

Conclusion

Motivation



Construction sites are dangerous

- Falls are the No-1 cause of death on construction sites
- Many falls go unseen – no one reacts in time



Real-time detection saves lives

- Stop machines, call help immediately



Why vision-based?

- Cameras are cheaper, always on
- Sensors are limited, uncomfortable
- Challenge: occlusion & speed

Problems in Existing Work

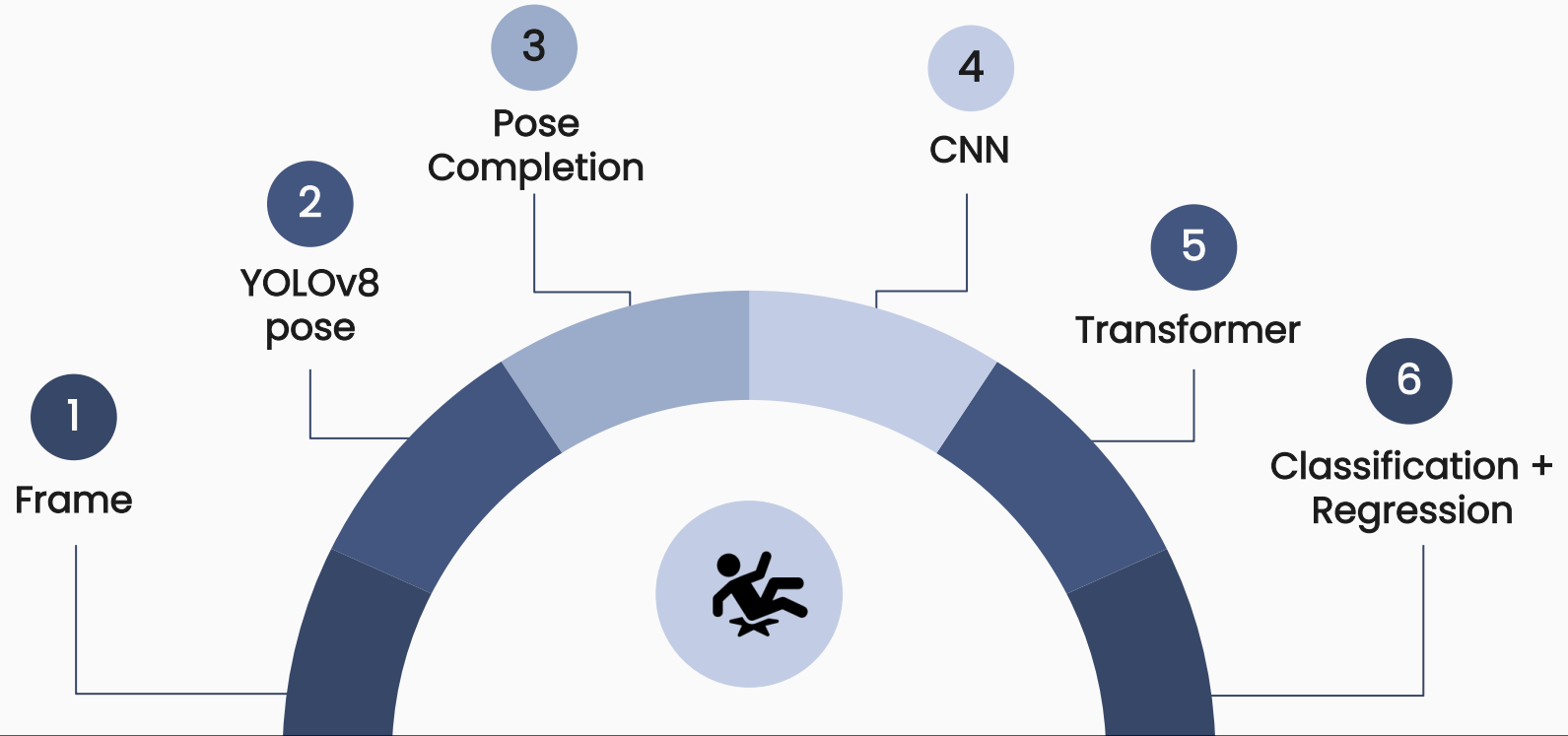
- 1 Sensor-based systems: expensive, non-scalable
- 2 Old vision models: slow, not real-time
- 3 Occlusion not handled
- 4 Pre-fall not predicted

Our Objective

- 1 Real-time detection
- 2 Handle occlusion
- 3 Predict fall before it happens (Pre-fall)
- 4 Estimate time-to-fall (regression)
- 5 Edge-compatible (Jetson, etc.)



System Overview Diagram



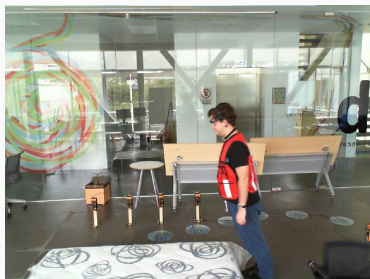
Dataset Use

UP-Fall – only frames

Publicly available

17-folders, one per subject:

11-subfolders, one per activity:



Custom video dataset

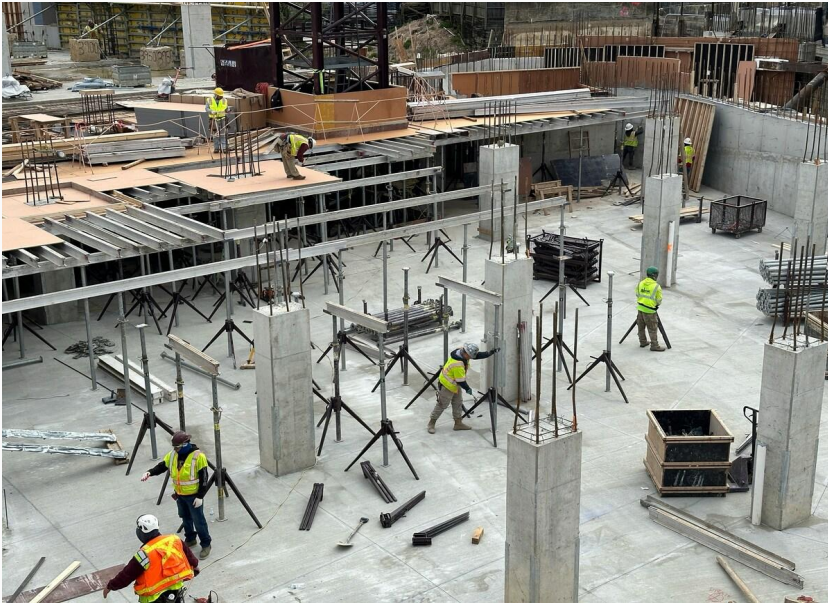
Multiple people in a frame (multi-person scenarios)

PPE gear (helmets, vests, etc.)

Various fall types with occlusion

Pre-fall behavior recorded explicitly

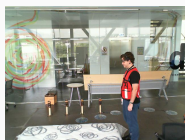
Pose Completion



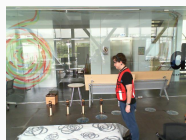
Problem: Occlusion = missing joints $\Rightarrow$ **Solution:** MLP or AutoEncoder fills missing keypoints $\Rightarrow$ Lightweight for real-time

Sequence Modeling (CNN)

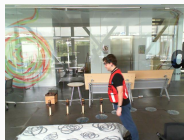
- ✓ Detects short-term motion patterns
- ✓ Works on a sliding window of pose vectors
- ✓ Very fast – ideal for real-time systems



Frame 1



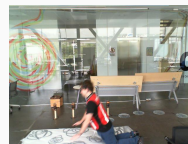
Frame 2



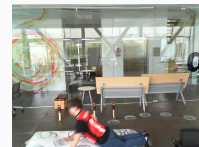
Frame 3



Frame 4



Frame 5



Frame 6



Frame 7

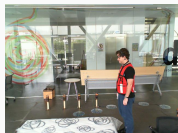
Used as a motion window for CNN

Temporal Attention (Transformer Encoder)

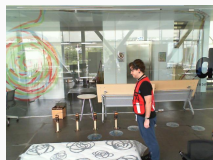
Captures important frames in time

Focuses on motion before fall

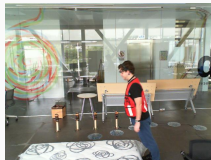
Efficient attention head



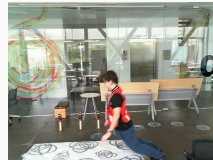
Frame 1



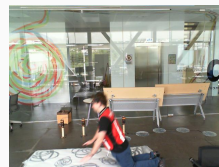
Frame 2



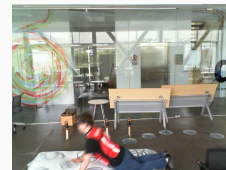
Frame 3



Frame 4



Frame 5



Frame 6



Frame 7

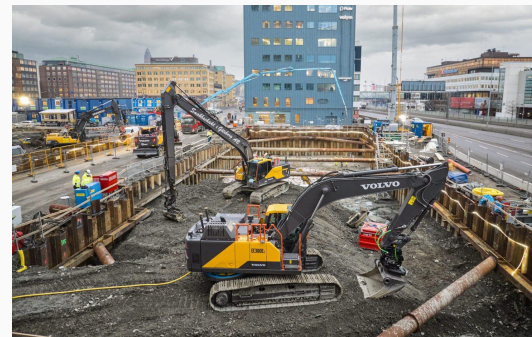
Dual Output Head

Output 1	Output 2
Classification	Regression
Fall / Pre-fall / No fall	Time-to-fall in seconds (tff = 1.45)

Fall tff = 1.45



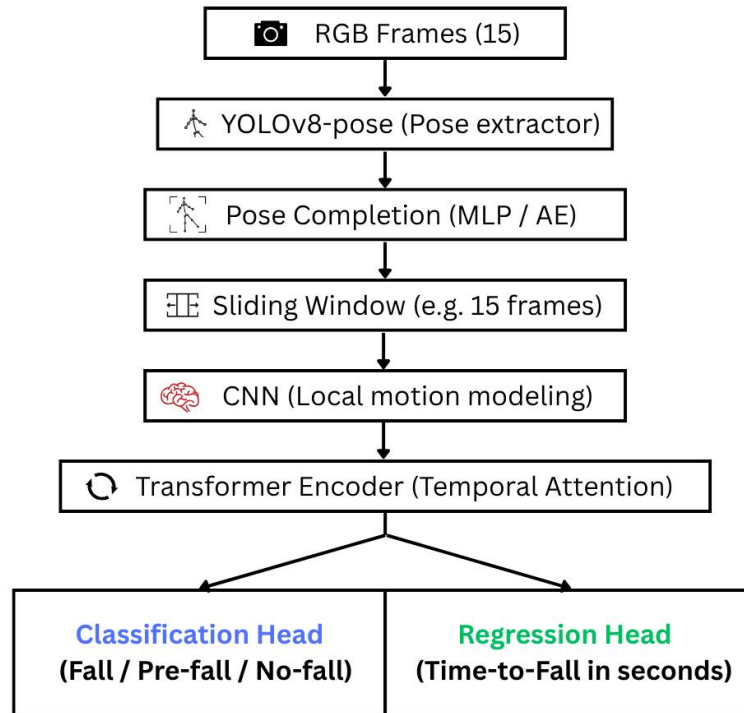
Dual Why Is Regression Important?



Regression predicts *"how many seconds until a fall occurs"* - *(e.g., "1.7 seconds to fall")

- Enables early warning systems: **alarm, drone, emergency response**
- Helps take action before the actual fall happens
- Offers a precise estimate of how close the person is to falling

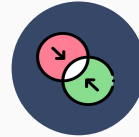
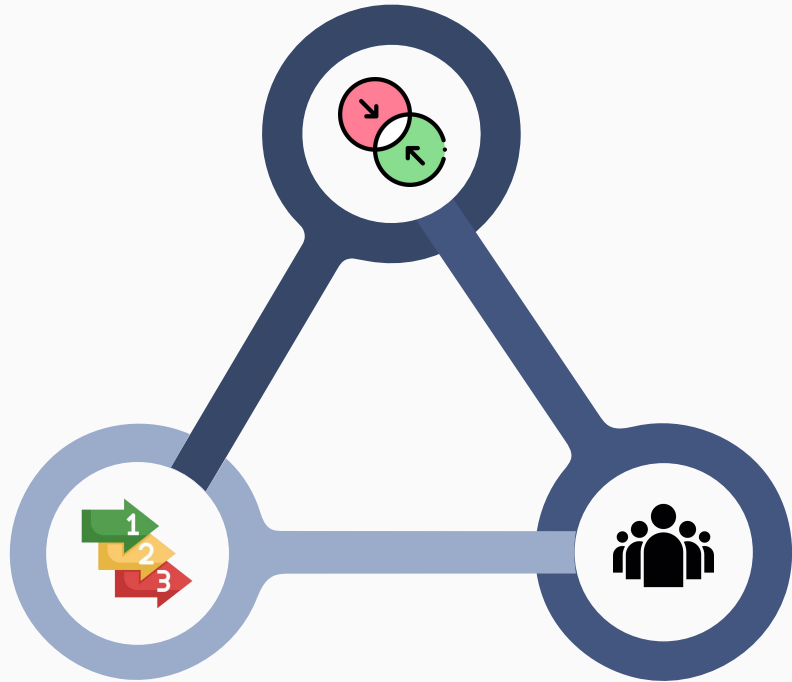
Model Architecture Overview



Comparative Table

Model	Occlusion	Pre-fall	Regression	Attention	Real-time
YOSAP	✗	✓	✗	✗	✗
TCNTE	✗	✗	✗	✓	✓
TST	✗	✗	✗	✓	✗
Ours	✓	✓	✓	✓	✓

Model Innovation Summary



First

Combine pose completion +
attention + TTF regression



Multiple people

Multi-person ready, real-time capable



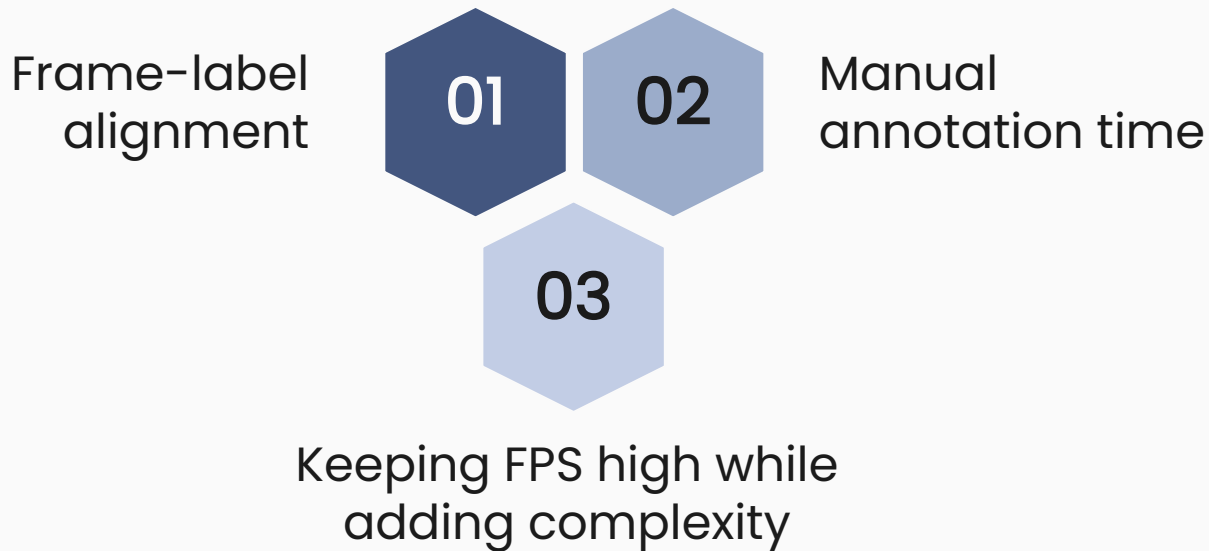
3 Stage fall detection

Fall prevention, not just detection

Development Progress

Task	Status
Dataset exploration	In progress (analyzing UP-Fall, reviewing options)
Pose extraction (YOLOv8-pose)	Planned
Pose completion	Studying techniques (MLP, AutoEncoder)
Annotation (Pre-fall, TTF)	To be designed
Model structure comparison	CNN, TCN, Transformer analysis ongoing
Training	Not started
Evaluation	Not started

Technical Challenges



Conclusion & Final Thoughts

We proposing a **real-time vision-based fall detection system** tailored for **construction site environments**

Our model handles:

- **Occlusion** via Pose Completion
- **Short-term motion** via CNN
- **Temporal patterns** via Transformer
- **Fall risk** via Classification + Regression

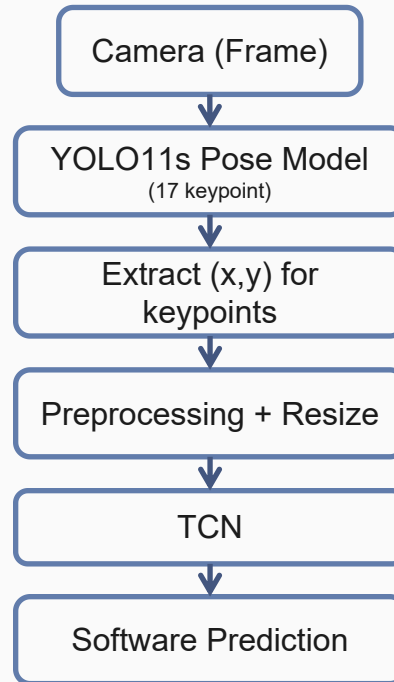
 *Currently under development – focusing on:*
→ *Annotation, model training, real-time optimization*

Progress Presentation

Real-time Fall Detection using Pose & TCN

Abbos Aliboev

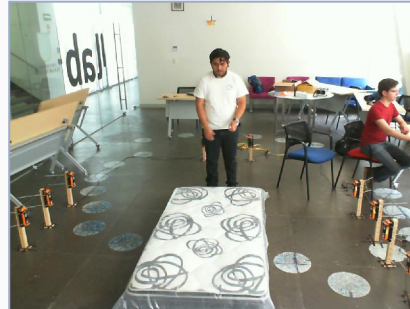
Method



1. Dataset Preparation

FALL-UP DATASET

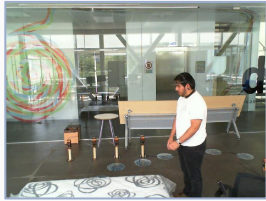
- Each activity folder has around 1000 frames.
- There are over 100,000 frames in total.
- We selected about **5000** frames from 5 activities.



Subject3	Activity3	Trial3
Subject4	Activity4	
Subject5	Activity5	
Subject6	Activity6	
Subject7	Activity7	
Subject8	Activity8	
Subject9	Activity9	
Subject10	Activity10	
Subject11	Activity11	
Subject12		
Subject13		
Subject14		
Subject15		
Subject16		
Subject17		

1. Dataset Preparation

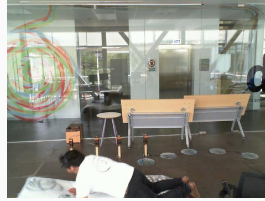
- ✓ Labeled each frame: no_fall, pre_fall, fall
- ✓ Created CSV with labels per frame



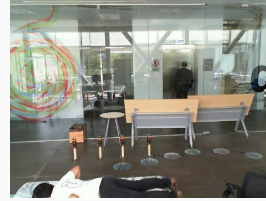
no-fall



pre-fall



pre-fall



fall

Example:

1 folder - 189 frame

1-15 **no-fall**

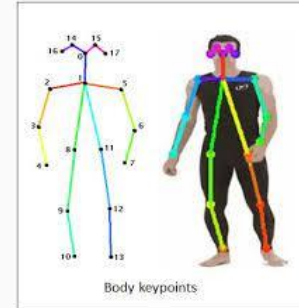
16-25 **pre-fall**

26-189 **fall**

```
fall_labels_all_activities.csv > data
filename,label
Subject1\Activity1\1\1.png,no_fall
Subject1\Activity1\1\2.png,no_fall
Subject1\Activity1\1\3.png,no_fall
Subject1\Activity1\1\4.png,no_fall
```

2. Pose Extraction

- ✓ Used **YOLOv11 pose** model
- ✓ Extracted **17 keypoints** per frame
- ✓ Saved pose data into CSV (x1,y1,...,x17,y17)
- ✓ Used as input features for model



Example:

```
pose_features_11s.csv > data
filename,x1,y1,x2,y2,x3,y3,x4,y4,x5,y5,x6,y6,x7,y7,x8,y8,x9,y9,x10,y10,x11,y11,x12,y12,x13,y13,x14,y14,x15,y15,x16,y16,x17,y17
Activity1/1/1.png,419.07391357421875,212.07620239257812,424.5980224609375,206.24595642089844,0.0,0.0,447.7557678222656,205.2068328857422,
Activity1/1/2.png,418.9844970703125,211.79025268554688,424.6172790527344,206.43365478515625,0.0,0.0,447.0237121582031,205.66717529296875,
Activity1/1/3.png,419.0434265136719,211.59249877929688,424.6343994140625,206.23388671875,0.0,0.0,446.9205322265625,205.54881286621094,0.0,
Activity1/1/4.png,418.0633239746094,210.30819702148438,424.0849609375,204.3818359375,0.0,0.0,445.5086669921875,205.11892700195312,0.0,0.0,
Activity1/1/5.png,416.359619140625,209.51808166503906,421.9396667480469,204.43203735351562,0.0,0.0,443.8798522949219,205.90512084960938,0.0,
Activity1/1/6.png,415.46807861328125,209.50616455078125,421.2569580078125,204.3729248046875,0.0,0.0,442.829833984375,206.240966796875,0.0,
Activity1/1/7.png,415.3357849121094,210.12057495117188,421.0819396972656,204.9110107421875,0.0,0.0,442.702392578125,206.31039428710938,0.0,
Activity1/1/8.png,414.76708984375,210.4777069091797,420.5155029296875,205.27928161621094,0.0,0.0,442.1677551269531,206.738525390625,0.0,0.0,
Activity1/1/9.png,414.5296936035156,210.8468017578125,420.33026123046875,205.5425262451172,0.0,0.0,442.2242126464844,206.46795654296875,0.0,0.0,
```

3. Pose Verification & Cleaning

- ✓ Detected **missing** poses (zero vectors)
- ✓ Total missing frames: 105 out of 5665
- ✓ Marked such frames for cleaning
- ✓ Ensures clean input for model

Example:

Activity2/5/161.png,,
Activity2/5/162.png,,
Activity2/5/163.png,,

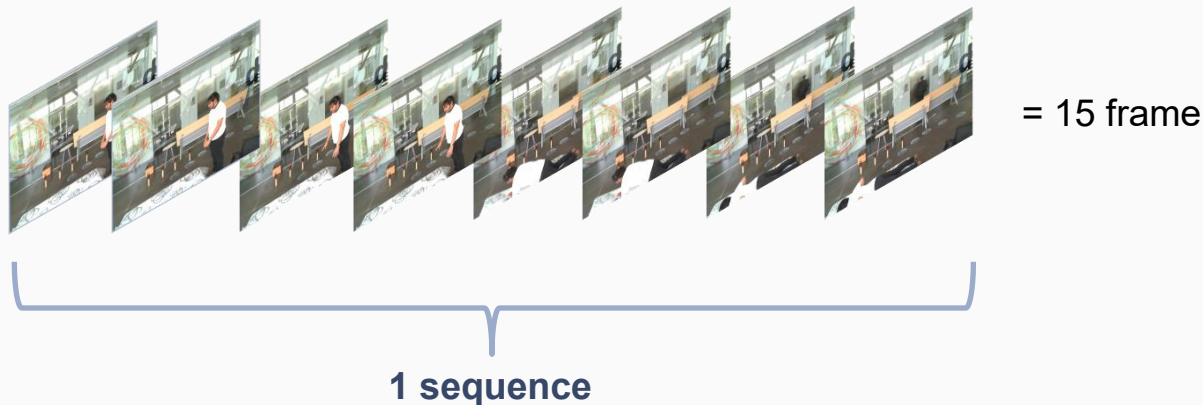


Activity2/5/161.png,0.00,0.00,0.00,0.00,0.00,0.00,0.00,0.00
Activity2/5/162.png,0.00,0.00,0.00,0.00,0.00,0.00,0.00,0.00
Activity2/5/163.png,0.00,0.00,0.00,0.00,0.00,0.00,0.00,0.00

4. Sequence Dataset (Sliding Window)

- ✓ Created **15 frame** sequences with **stride=1**
- ✓ Each sequence labeled by priority: **fall > pre_fall > no_fall**
- ✓ Total sequences generated: **144,034**
- ✓ Saved to: **fall_sequences_priority_label.csv**

Example:

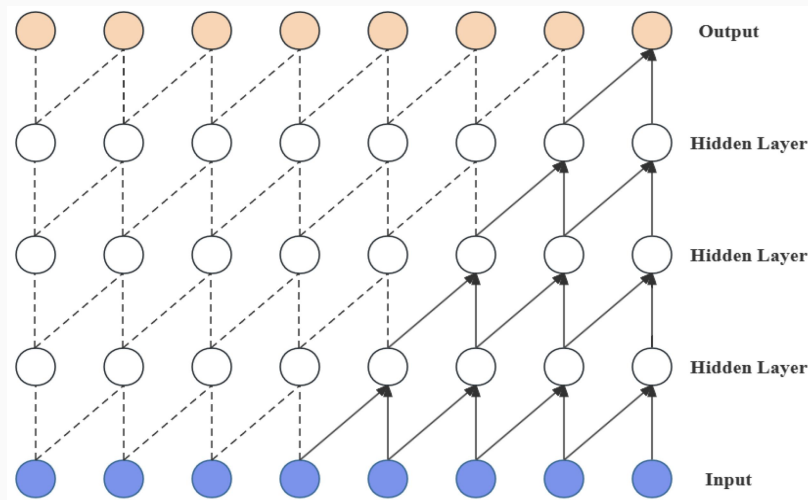


5. TCN Model Training

- ✓ Trained TCN model with **10 epochs**
- ✓ Input: (15, 34) per sequence
- ✓ Achieved **91%** overall accuracy
- ✓ Model saved: tcn_model.pth

Results:

Epoch 01	Loss: 1.2130	Accuracy: 88.40%
Epoch 02	Loss: 0.3262	Accuracy: 89.15%
Epoch 03	Loss: 0.2800	Accuracy: 89.96%
Epoch 04	Loss: 0.2648	Accuracy: 90.50%
Epoch 05	Loss: 0.2568	Accuracy: 91.41%
Epoch 06	Loss: 0.2467	Accuracy: 91.10%
Epoch 07	Loss: 0.2448	Accuracy: 88.50%
Epoch 08	Loss: 0.2393	Accuracy: 87.49%
Epoch 09	Loss: 0.2336	Accuracy: 91.46%
Epoch 10	Loss: 0.2294	Accuracy: 91.13%



Structure of TCN

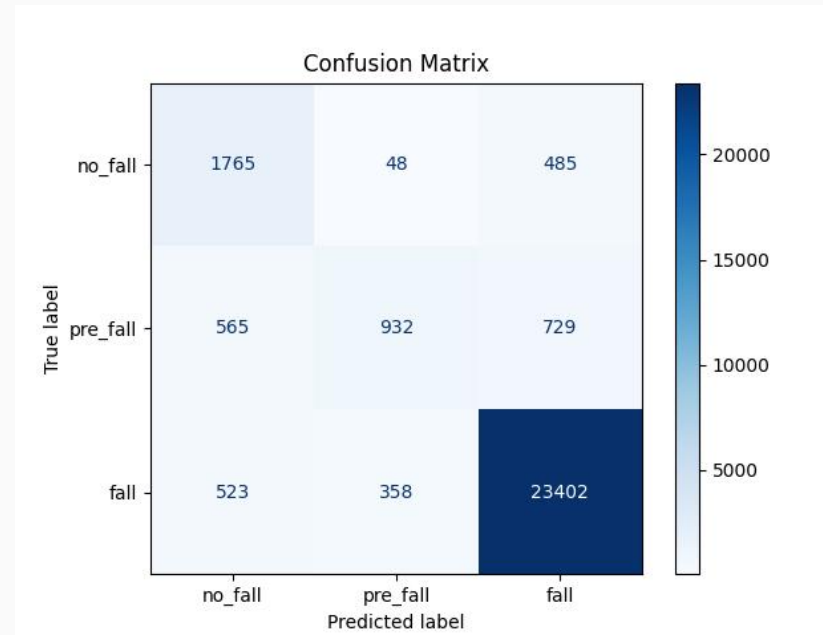
6. Evaluation Results

Class-wise performance:

- **Fall** → F1: 0.96, Precision: 0.95
- **Pre-fall** → F1: 0.57 (needs improvement)
- **No-fall** → F1: 0.70

	precision	recall	f1-score	support
no_fall	0.6186	0.7681	0.6853	2298
pre_fall	0.6966	0.4187	0.5230	2226
fall	0.9507	0.9637	0.9572	24283
accuracy			0.9060	28807
macro avg	0.7553	0.7168	0.7218	28807
weighted avg	0.9046	0.9060	0.9019	28807

Next step: Improve pre-fall detection



Demo



If a person stands too close to the camera, it falsely detects a fall

Demo - 2



Average FPS: **62.77**

Improved logic and better fall detection handling.

Summary

- ✓ Successfully built labeled **dataset**
- ✓ Performed **pose estimation** using YOLO
- ✓ Created **temporal sequences** for training
- ✓ Trained **TCN** model with good performance

Future work:

- Improve Pre-fall Detection
- Add Person Detection Filtering
- Distance-Aware Logic
- Pose Quality Assessment
- Balanced Dataset
- Visual Alert System
- Multi-person Fall Detection

Thanks!

Do you have any questions?

www.abbosaliboev.site